Channel Pipeline: Master Plan, Setup Spec and Claude Code Prompts

Prepared 31 Oct 2026 for a faceless, quality-first explainer channel: YouTube Shorts, Instagram Reels and Facebook Reels first, long-form later. Paths use forward slashes; PowerShell accepts them.

0. How to use this document

1. Today: do the prep checklist (Part 14) and answer the open questions (Part 15).
2. Tomorrow, after buying Pro: create `C:/channel`, save this document inside it as `docs/MASTER_PLAN.md`, install Claude Code, open it in that folder, then run Prompt 1, Prompt 2 and Prompt 3 (Part 17) in separate sessions. Do not paste them together: small sessions are easier to debug and kinder to usage limits.
3. No plan can guarantee that nothing breaks. This plan catches problems early instead: a `doctor` script, a staged test video, validators before every slow step, and git commits at each milestone.
4. Package names, commands, prices, usage limits and platform rules change. The prompts tell Claude Code to verify them against official docs before relying on them.

1. Principles

- Quality over volume. Aim for 2 to 3 strong videos a week. You approve at four gates.
- Brand identity is written down in `brand/`, not improvised per video.
- Every stage reads and writes files. Any stage can be re-run alone. Nothing depends on chat memory, so a usage-limit cutoff never loses work.
- Nothing enters a video without a license record.
- Deterministic work (TTS, alignment, mixing, checks, rendering) runs as scripts. Claude's reasoning goes to research, writing, direction and judgement.
- Fail early: validators stop the pipeline before a slow render, not after.
- Your taste decides what ships. Audience data tells you which style works.

2. Assumptions (confirm in Part 15)

- Windows 10/11 64-bit, ASUS TUF A15, RTX 3050 with 4 GB VRAM, native Windows (not WSL).
- English narration with Kokoro. Format first: 1080x1920, 30 fps, 30 to 60 seconds.
- Claude Pro plan, Claude Code installed natively. No paid APIs, no AI image generation, no downloaders for copyrighted media.
- Whisper and Kokoro run on CPU first. GPU is an optional speed-up after the tests pass.
- You upload manually at first. Auto-upload is a later project.

3. Pipeline stages and gates

Each video lives in `videos/<id>/` with a `state.json` recording each stage's status and input hashes, so unchanged stages are skipped.

- S0 Intake: you give a title or idea. Output `00_brief.md` (title, angle, audience, target length, series, style-variant tag).
- S1 Research: `01_research.md` and `sources.json`. Primary sources preferred, at least two independent sources per key claim, dates and numbers checked, a list of what is uncertain, and one sentence stating the unique angle. **Gate 1: you approve.**
- S2 Script: `02_script.md` with three hook options, narration split by scene, word count checked against target length (about 140 to 160 words per minute), new names added to `brand/pronunciations.json`. **Gate 2: you approve.**
- S3 Storyboard: `03_storyboard.json` (schema-validated) and `03_assets_needed.md`. Per scene: narration, visual type, on-screen text, animation, transition, SFX ids from the catalog, music mood, asset needs with status. **Gate 3: you approve, audition the SFX picks, and supply anything missing.**
- S4 Assets: fetch or add files, record each in `licenses.json`, run `validate_assets`. It fails if any file is missing or unlicensed.
- S5 Voice: Kokoro per scene to `audio/scene_XX.wav`. Text normalised first, cached by hash of text+voice+speed, resampled to 48 kHz.
- S6 Alignment: faster-whisper word timestamps. The script text is the truth; Whisper only supplies timing. A mismatch report is written if they disagree.
- S7 Timeline: scene lengths computed from real audio durations (rounded up to frames plus padding) into `timeline.json`.

- S8 Mix: voice, ducked music, SFX at cue frames, short fades, loudness normalised, saved as `audio/mix.wav`.
- S9 Build and draft render: Remotion reads `timeline.json` and writes `renders/draft_vN.mp4` at reduced scale for speed.
- S10 Automated QA: `qa/report.json` plus stills (Part 9).
- S11 Claude reviews the stills, fixes layout problems, repeats up to 3 times, then reports what is left.
- **Gate 4: you watch and listen to the draft** and write `feedback_vN.md`.
- S12 Revision: map feedback to script or storyboard edits and re-run only the affected stages.
- S13 Final: full-resolution `renders/final.mp4`, `publish.md` (3 titles, description, hashtags, attribution lines, AI-disclosure decision), copy to `exports/`.
- S14 Log: add a row to `analytics/experiments.csv`; update metrics at 48 hours and 7 days.

4. Toolchain and dependencies

Install with winget where possible. IDs below are typical; Claude must confirm them with `winget search` first.

System tools (verify command):

- Node.js LTS (`OpenJS.NodeJS.LTS`): `node --version`, `npm --version`
- Git (`Git.Git`): `git --version`
- Python 3.11 (`Python.Python.3.11`): `py -3.11 --version`. Pin 3.11; the newest Python often lacks PyTorch wheels.
- FFmpeg with ffprobe (`Gyan.FFmpeg`): `ffmpeg -version`, `ffprobe -version`
- espeak-ng (Kokoro needs it on Windows): `espeak-ng --version`. If Kokoro cannot find it, set the library-path environment variable named in the error.
- Claude Code (native installer): `claude --version`, `claude doctor`
- NVIDIA driver (already present, update it): `nvidia-smi`

Python packages (inside `.venv`, pinned in `requirements.txt`): `kokoro`, `soundfile`, `faster-whisper`, `torch` (CPU build first), `numpy`, `librosa` (BPM and beat grid), `pyloudnorm`, `jsonschema`, `Pillow`.

Remotion project (TypeScript; commit `package-lock.json`, use `npm ci`): `remotion`, `@remotion/cli`, and as needed `@remotion/transitions`, `@remotion/shapes`, `@remotion/paths`, `@remotion/captions`, `@remotion/layout-utils`, `@remotion/fonts`, plus `d3-geo`, `topojson-client`, `world-atlas` for maps. Confirm exact names on remotion.dev/docs. Install the official Remotion skill for Claude Code from the Remotion docs. Remotion is free for individuals and companies of up to 3 people; recheck its license page if your team grows.

Models (download once into `cache/hf`): Kokoro-82M (Apache-licensed weights) and Whisper `small` through faster-whisper (try `medium` only if accuracy is poor). Make sure headless Chrome for Remotion is downloaded during setup, not on the first render.

Fonts: open-licensed (OFL) fonts downloaded into `brand/fonts/`. Never depend on a network font at render time.

Do not install: video downloaders, image-generation tools, or anything that scrapes copyrighted media.

5. Windows hardening (Claude asks before each)

- Enable the Windows long-paths setting (admin) and run `git config --system core.longpaths true`.
- `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` so venv activation scripts run.
- Keep the project at `C:/channel`: short, no spaces, outside OneDrive or Dropbox.
- Set `PYTHONUTF8=1` and open every file with `encoding='utf-8'`. Windows default encodings break names with accents.
- `.gitattributes`: LF for code, binary for media. `.gitignore`: `node_modules`, `.venv`, `cache`, `renders`, `.env`.
- Open a new terminal after every install; old windows keep a stale PATH. `doctor` rechecks it.
- Windows Defender: optionally exclude `remotion/node_modules` and `cache` for faster installs and renders. Your call.
- Power: plugged in, best-performance mode, sleep and hibernate off during renders, laptop on a hard surface or cooling pad.
- Disk: keep 60 GB or more free on the SSD. A `clean_cache` script removes old drafts and caches, never finals.

- Do not run Claude Code as administrator. For installer prompts, you click Yes.
- Remotion: set concurrency explicitly (start at 2), render drafts at reduced scale, avoid stacks of blur and shadow effects.

6. Folder structure

```
C:/channel
  CLAUDE.md
  README.md
  SETUP_REPORT.md
  docs/        MASTER_PLAN.md, PLATFORM_NOTES.md
  brand/       brand.md, voice.json, pronunciations.json, palette.json,
fonts/, intro/
  config/      pipeline.json (fps, size, loudness targets, voice, paths)
  schemas/     storyboard, timeline, licenses, catalog, sources, state (JSON
Schema)
  pipeline/    doctor.py, run_stage.py, normalize_text.py, tts.py, align.py,
               timeline.py, mix_audio.py, catalog_assets.py, find_sfx.py
(later),
               validate.py, validate_assets.py, stage_assets.py, qa_check.py,
               clean_cache.py
  remotion/    src/components, src/theme, src/Root.tsx, public/staged/<video-
id>/
  library/
    sfx/       files, catalog.json, core_set.json
    music/     files, catalog.json, SOURCES.txt
    logos/ footage/ images/ maps/
    licenses.json   one record per asset
  videos/<yyyy-mm-dd_slug>/
    00_brief.md  01_research.md  sources.json  02_script.md
    03_storyboard.json  03_assets_needed.md  timestamps.json  timeline.json
    state.json  audio/  qa/  renders/  feedback_vN.md  publish.md
  exports/     final video plus publish package per video
  analytics/   experiments.csv
  cache/       hf/ tts/ align/   (safe to delete)
  logs/
  tests/       fixtures and smoke tests
  .venv/
```

Remotion only loads files from its `public/` folder, so `stage_assets.py` copies just the files a video needs into `remotion/public/staged/<video-id>/` and the composition reads them with `staticFile()`.

7. Data contracts (JSON Schemas in `schemas/`)

- Storyboard scene: id, narration, visual (type and props), onScreenText[], transition, sfx[] (catalog id, anchor word index, gain dB), music (mood, track id), assets[] (id, status: have / fetch / needed from you).
- Timeline: fps, width, height, scenes[] (id, startFrame, durationFrames, audio file, words[] with start and end frame), sfxCues[], music track.
- License record: assetId, file, sha256, sourceUrl, author, license, licenseUrl, attributionRequired, attributionText, retrievedOn, approvedBy (claude-checked or user-approved), notes.
- Catalog entry (sfx and music): id, file, durationSec, sampleRate, peakDb, loudnessLufs, leadingSilenceMs, tags[], description, useFor[], energy, bpm and key (music), core (true/false), licenseRef.
- Source: id, url, title, publisher, date, retrievedOn, claimsSupported[].
- State: per stage status, input hash, outputs, finishedAt.

8. Assets, licensing and finding the right SFX

Rules:

- Allowed: public domain or CC0; licenses allowing commercial use with no attribution; licenses allowing it with attribution (the attribution text is added to `publish.md` automatically); files you own or bought. Held until you decide: unknown license, non-commercial, no-derivatives, and any commercially released music.
- Every asset gets a record in `licenses.json` (source URL, license, date, file hash). `validate_assets` blocks the build if a referenced asset has no record.
- Starter source list: Wikimedia Commons, Natural Earth, NASA media, Pexels, Pixabay, Internet Archive (check each item), and official press or brand kits for logos. Claude checks the license on each item's own page; you approve anything ambiguous.
- Logos: only from official press kits or Commons, used to identify the company, never altered, never implying endorsement. Fall back to a text wordmark.
- Maps: Natural Earth or world-atlas. OpenStreetMap data needs attribution.

- Footage: transcode to H.264, yuv420p, constant 30 fps before use, because variable frame rate causes sync drift. Use Remotion's OffthreadVideo.
- Images: resize to the largest size needed. No AI-generated images.
- Music you supply: record the source and license of every track in `library/music/SOURCES.txt`. Commercially released songs lead to blocked or muted videos.

Finding the right SFX. Claude cannot hear audio. It matches text to text: file names, folders, tags, descriptions and measured numbers. With a good catalog it does this well, and for a few hundred files no extra technology is needed.

1. Phase 1, catalog. `catalog_assets.py` scans the folders, measures duration, peak, loudness, sample rate and silence, takes tags from folder and file names, and lists files lacking a useful description in `catalog_todo.md`. You write one line for each; you can listen and Claude cannot.
2. Core set. Pick 40 to 80 sounds that define your brand. Claude prefers the core set unless you approve otherwise. This also makes your sound recognisable.
3. At Gate 3, a generated `qa/sfx_audition.html` page shows each scene next to play buttons for its chosen sounds, so you catch wrong picks in a minute.
4. Phase 2, only if needed (library above roughly 300 to 500 files, or hand-tagging is too slow): add a semantic layer. Compute audio-text embeddings once per file with a CLAP-type model (check license and size first; CPU is fine), keep them in a small local index (numpy or SQLite; a vector database is overkill), and expose `find_sfx.py 'soft whoosh for map zoom' --top 5 --max-dur 1.2`. Claude then chooses among 5 candidates instead of reading hundreds of entries.
5. Music: add BPM and beat grid (librosa), energy, mood tags and loop points so cuts can land on the beat.
6. Track usage counts so one sound is not overused. Repetition builds a brand; fatigue loses viewers.

9. QA specification

Automated (`qa_check.py`, errors stop the pipeline):

- ffprobe: 1080x1920, constant 30 fps, H.264, yuv420p, AAC 48 kHz; audio and video durations match the timeline within one frame.

- Audio: integrated loudness near -14 LUFS and true peak at or below -1 dBTP (common social targets; verify per platform), no clipping, no unplanned silence over 1.5 s, music bed roughly 15 to 20 dB under voice as a starting point.
- Video: no unplanned black frames; no text outside safe zones (start with 60 px sides, 150 px top, 250 px bottom, then tune on real phones); text measured with layout-utils before rendering.
- Captions: every script word appears; sensible line lengths; nothing spills past its scene.
- Assets: every referenced file exists and has a license record; attribution lines are in `publish.md`.
- Stills: first frame, both sides of every scene boundary, and mid-scene, saved to `qa/stills/` and reviewed by Claude for overlaps, cut-off text, wrong assets and empty frames.

Human (Gate 4): watch with sound on a phone. Check the hook, pacing, pronunciation, volume balance, factual claims against `sources.json`, and brand consistency.

Platforms: before the first upload, verify current maximum length and file size for Shorts, Reels and Facebook Reels (30 to 60 s should be safe everywhere), caption limits, and each platform's synthetic-content disclosure setting. Re-check policies every quarter. YouTube evaluates channels as a whole, so keep scene mix, structure and angles varied.

10. Failure-mode checklist

Environment

- Stale PATH after an install: open a new terminal; `doctor` checks.
- Several Pythons: always use `py -3.11` and the venv.
- Torch and CUDA mismatch: CPU build first; GPU later as a separate step.
- espeak-ng not found: set the library path variable; `doctor` does a short Kokoro render.
- Model or Chrome download blocked or interrupted: retry, keep `cache/hf`, `doctor` verifies the files.
- Flaky npm install: lockfile, `npm ci`, long paths on, antivirus exclusions.

Audio

- Kokoro outputs 24 kHz: resample once to 48 kHz.
- Numbers, dates, abbreviations and names mispronounced: normalise text, keep a pronunciation dictionary (respell phonetically), listen at Gate 4.

- Long text: split at sentence boundaries; keep pauses between scenes consistent.
- Clicks, clipping, loudness drift: short fades, limiter, loudness normalisation on every mix.
- Stale cached audio after a script edit: the cache key includes text, voice and speed.

Sync

- Scene length comes from real audio, not estimates; round up to frames and add padding.
- Captions use script text; timing comes from Whisper; trim leading silence before aligning.
- Variable-frame-rate footage: transcode first.
- Put the final audio inside the Remotion render, or verify durations after muxing.

Visual

- Fonts missing at render: load local fonts and wait (delayRender) until they are ready.
- Text overflow: measure before rendering.
- Flicker or non-repeatable output: use seeded `random()`; never `Math.random` or `Date.now`.
- Memory crashes: concurrency 2, reduced-scale drafts, smaller images, fewer stacked blurs.
- SVG logos with external references or live text: outline the text and inline everything.

Assets and legal

- Unknown licenses, forgotten attribution, misused logos, music with copyright claims, footage of identifiable people without clear permission, maps without attribution: handled by the rules in Part 8.

Content

- Hallucinated or stale facts: `sources.json`, the two-source rule, dates on every number, and you read the research.
- On-screen numbers that differ from narration: QA compares storyboard text with the script.
- Claims about real people or companies: documented facts with sources only; no accusations.

Process

- Usage-limit cutoff mid-session: stages are resumable through `state.json`; one stage per session; `/clear` between stages.
- Context bloat: keep CLAUDE.md short, never paste large files, read only what the stage needs.
- Claude edits the wrong thing: git commit before every stage; allow-list project commands; deny deletes outside the project.
- Lost work: back up `brand/`, `library/`, `videos/` and `exports/` weekly to an external drive or cloud (skip `node_modules`, `.venv`, `cache`).
- Secrets: none needed now; `.env` is git-ignored if that changes.

11. Brand bible and experiments

`brand/brand.md` sections: name and one-line promise; audience; tone (3 adjectives, 3 words you never use); palette (3 to 4 hex values and usage rules); 1 or 2 fonts; shape language; caption style; motion rules (easing, durations, one signature transition); intro or outro sting (1 second or less); narrator voice and speed; writing rules (sentence length, humour level, hook formula, structure, ending line); sound rules (core SFX set, music moods, loudness); do-not list.

Voice audition: the same neutral paragraph in 6 voices labelled A to F blindly, then the top 2 at three speeds. Style exploration: the same 10-second scene in 3 visual styles. Both are in Prompt 3.

Experiments: change one variable per round (voice, pacing, hook type, scene mix, music mood). Use 3 to 4 videos per variant on similar topics and judge after 12 to 15 videos. `analytics/experiments.csv` columns: video id, date, platform, style variant, hook type, length, topic type, early swipe-away or hold rate, average percent viewed, shares, saves, follows gained, notes.

12. Usage plan

- Setup and scaffolding are the heaviest sessions. Start them in a fresh usage window.
- One stage per session, `/clear` between them. The files on disk carry the state.
- After the test video, check Settings > Usage and note what setup cost. After the first 3 real videos, extrapolate to your monthly target. If limits bite: wait, enable extra usage, or move up a plan.

- Use the lighter model for routine scripts and QA; save the stronger one for research and script writing.

13. Roadmap

- Phase 0 (today): prep checklist.
- Phase 1 (day 1): Prompt 1. Setup finishes and `doctor` passes.
- Phase 2 (day 1 to 2): Prompt 2. Pipeline skeleton and test video.
- Phase 3: Prompt 3. Voice audition, style exploration, fill the brand bible.
- Phase 4: grow the component library to about 10 scene types: hook text, kinetic captions, stat counter, bar or line chart, timeline, map pan and zoom, image with slow zoom, comparison split, quote card, name or logo reveal, plus brand transitions.
- Phase 5: first 3 real videos through every gate. Fix pipeline pain points.
- Phase 6: style tests; review at 12 to 15 videos.
- Later, long-form: 16:9 compositions, chapters, more assets, longer renders and more disk, sources shown on screen, a thumbnail step.

14. Prep checklist (today)

- Update Windows and the NVIDIA driver, then restart.
- Free at least 60 GB on the SSD.
- Create `C:/channel`, then `library/sfx` and `library/music` inside it, and copy in your SFX folder and tracks.
- Write the source and license of each music track in `library/music/SOURCES.txt`. If you cannot, do not use the track.
- Pick a working channel name, a niche or tone direction, and 3 candidate topics.
- Note your CPU, RAM, Windows version and free disk space (Settings > System > About).
- Save this document as `C:/channel/docs/MASTER_PLAN.md`.
- Make sure your internet is stable; models and packages are large.

15. Open questions

- CPU model, RAM, Windows version, free disk space?
- Narration language: English only?

- Channel name, niche and tone direction?
- How many SFX files, where did they come from, and under what license?
- Source and license of the music tracks?
- Do you want auto-upload later, or manual uploads for good?

16. CLAUDE.md text (Prompt 1 writes this file)

```
# Channel pipeline: rules for Claude
Project: faceless, quality-first explainer videos (Shorts/Reels). Windows,
PowerShell, native.
Full spec: docs/MASTER_PLAN.md (read only the part you need).

## Always
- One stage at a time. Read videos/<id>/state.json first. Write outputs to
files. Never rely on chat memory.
- Stop at gates 1-4 and wait for my approval. Never skip a gate.
- Use py -3.11 and .venv only. Never install into global Python. Never run as
admin.
- Work only inside this folder. Ask before deleting anything, downloading
over 500 MB, or changing system settings.
- Commit to git before each stage and after each approved gate.

## Content
- Facts need sources.json entries (2 independent sources for key claims,
dates on numbers). Mark uncertain claims. No accusations about real people or
companies beyond documented facts.
- Follow brand/brand.md for tone, voice, palette and motion. Do not invent
new styles per video.

## Assets
- Use only files in library/ with a record in licenses.json. Unknown license:
do not use it, tell me.
- No AI-generated images. No commercially released music. Prefer SFX in
core_set.json and log SFX use.

## Technical
- 1080x1920, 30 fps. Scene length from audio. Captions from script text.
Seeded random only. Local fonts. Text measured before render. Footage
transcoded to constant 30 fps.
```

```
- Run validate.py and validate_assets.py before rendering and qa_check.py
after. Fix failures; after 3 failed attempts on one problem, stop and report.
- Verify package names and commands against official docs before installing.
Do not trust memory.
```

17. Claude Code prompts

Run each in a fresh session, from inside `C:/channel`.

Prompt 1: environment setup and doctor

```
You are setting up a video production pipeline on this Windows PC. The
current folder is the project root (C:/channel). First read
docs/MASTER_PLAN.md (Parts 2 to 6, 10 and 16). Follow it unless official docs
contradict it; if they do, follow the docs and note the difference in
SETUP_REPORT.md.

Rules
- Work only inside this folder, except installing system tools with winget.
- Ask me before: anything needing admin rights, any download over 500 MB,
changing system settings (long paths, execution policy, Defender exclusions),
or deleting anything.
- Before installing anything, verify package names, versions and commands
against official docs (Node.js, Remotion at remotion.dev/docs, Kokoro on
GitHub hexgrad/kokoro, faster-whisper, FFmpeg, espeak-ng, winget). Do not
rely on memory.
- Commit to git after each numbered step.

Steps
1. Read-only inspection: Windows version, CPU, RAM, free disk, nvidia-smi,
winget, and which of node, git, python 3.11, ffmpeg, ffprobe, espeak-ng
exist. Summarise it. Continue unless something blocks.
2. Install what is missing via winget (Node LTS, Git, Python 3.11, FFmpeg,
espeak-ng). Tell me whenever a new terminal is needed and re-check PATH
afterwards.
3. Apply the Windows hardening items in Part 5, asking for each.
4. Create the folder structure from Part 6, git init, .gitignore,
.gitattributes, config/pipeline.json, README.md, and write CLAUDE.md exactly
as in Part 16.
5. Create .venv with Python 3.11. Install kokoro, soundfile, faster-whisper,
CPU torch, numpy, librosa, pyloudnorm, jsonschema and Pillow with pinned
versions in requirements.txt. Make sure PYTHONUTF8=1 is set when the venv is
```

```
used.
6. Create the Remotion project in remotion/ (TypeScript, blank template).
Install the official Remotion skill and the packages from Part 4 that exist,
commit the lockfile, make sure headless Chrome is downloaded, and set
concurrency to 2.
7. Pre-download Kokoro-82M and the Whisper small model into cache/hf and
confirm they load.
8. Write pipeline/doctor.py (plus doctor.ps1) that checks: tool versions and
PATH, venv imports, espeak-ng found, a 1-second Kokoro render, Whisper on
that file, FFmpeg and ffprobe, a 2-second Remotion render, free disk space.
Print PASS or FAIL per check and save logs/doctor_<date>.txt. Run it and fix
failures. After 3 failed attempts on one problem, stop and ask me.
9. Write SETUP_REPORT.md: versions, what was installed, decisions, issues,
how to re-run doctor.

Stop after step 9. Do not start any video work.
```

Prompt 2: pipeline skeleton and test video

```
Read CLAUDE.md and docs/MASTER_PLAN.md Parts 3, 6, 7, 8 and 9. Build the pipeline skeleton.

1. JSON Schemas in schemas/ for storyboard, timeline, licenses, catalog, sources and state, plus pipeline/validate.py.
2. Scripts in pipeline/: normalize_text.py (numbers, dates, abbreviations, pronunciations), tts.py (Kokoro per scene, hash cache, resample to 48 kHz), align.py (faster-whisper word timestamps with the script as truth, plus a mismatch report), timeline.py (scene frames from audio), mix_audio.py (voice, ducked music, SFX cues, fades, loudness normalisation), catalog_assets.py (scan library/sfx and library/music into catalog.json and catalog_todo.md), validate_assets.py, stage_assets.py, qa_check.py (ffprobe specs, silence, black frames, loudness, duration match, stills at scene boundaries and mid-scene), and run_stage.py with a resumable state.json.
3. Remotion: theme with placeholder palette, local fonts and safe-zone constants; components HookText, KineticCaptions, TextCallout, StatCounter, Timeline, MapPanZoom (d3-geo and world-atlas), AssetImage with slow zoom, and a simple transition set; one composition ShortVideo that reads timeline.json. Fonts must load before rendering, random must be seeded, text must be measured before render.
4. Test video id 000_pipeline_test, 12 to 15 seconds, with a made-up neutral script (no real-world factual claims), one SFX and one music file from my library chosen via the catalog, no logos, synced captions. Run every stage end to end. Extract stills, review them yourself and fix layout problems (maximum 3 loops).
5. Output renders/draft_v1.mp4 and write qa/report.json and TEST_REPORT.md with time per stage, any memory problems seen, issues, and your recommendations.

Do no real research. Fetch nothing external except OFL fonts and world-atlas data.
```

Prompt 3: voice audition and style exploration

```
Read CLAUDE.md and brand/brand.md (template).

A) Voice audition. Use a neutral 60-word explainer paragraph with no factual
claims. Render it with 6 Kokoro English voices (mix of US and UK, male and
female) at normal speed. Label them A to F, keep the key in a separate file,
and generate voice_audition.html with play buttons. Wait for my choice. Then
render my top 2 at three speeds, wait again, and save the final choice to
brand/voice.json with a starter pronunciations.json.

B) Style exploration. Build the same 10-second scene (hook text, map move,
one statistic, captions) in 3 clearly different visual styles. For each,
describe palette, fonts, shapes and motion in two lines. Render all three,
produce comparison.png of matching stills, and write style_options.md. Wait
for my choice, then fill in brand/brand.md and brand/palette.json and commit.
```

Prompt 4: first real video (use later)

```
New video. Read CLAUDE.md and brand/brand.md. Topic: <title or idea>. Create
videos/<yyyy-mm-dd_slug>/00_brief.md and run stage S1 (research) only. Save
01_research.md and sources.json, state the unique angle and the
uncertainties, then stop for Gate 1. After I approve, I will ask for S2, then
S3, and so on, one gate at a time.
```

Pipeline v2: aiTrends 2D Studio + Remotion

Parts 18 to 27 were added on 1 Nov 2026 after inspecting the plugin package itself (aitrends-2d-studio v0.1.0, 61 files). Where they conflict with Parts 3, 6 and 7, Parts 18 to 27 win. Setup, Windows hardening, QA specs, failure modes and the brand bible (Parts 4, 5, 9, 10, 11) still apply.

18. What I verified, what I assumed, and where I disagree

18.1 Verified by reading the plugin files

- Package: a Claude plugin (`.claude-plugin/plugin.json` , v0.1.0, author aiTrends) with five skills: make-2d-video (pipeline, references and the `kit/`), new-video, video-styles, render-video, video-sound. **There is no LICENSE file and no license field**

anywhere in the package.

- Kit dependencies: playwright 1.56.0, three 0.169.0, three font packages (Fredoka, Nunito, Playfair Display); Python numpy, scipy, faster-whisper; ffmpeg; Node 18 or newer. `tools/doctor.py` checks all of it and `--install` fixes what it can.
- `render.mjs` options: --project, --from, --to, --out, --scale, --workers, --format png|jpeg, --crf, --preset, --gl, --sheet, --stills, --styles, --at, --style, --debug. A full render writes `build/video_silent.mp4`; a range render writes `build/preview.mp4` unless `--out` is given. Each worker is its own Chromium piping frames into ffmpeg (libx264, yuv420p, CRF 18 default); chunks are joined by stream copy. Default workers: half the CPU threads, max 8.
- Frame time is `t = frame / fps`, with fps read from `config.json` (default 24). So 30 fps is supported and deterministic. A range render covers frames round(from x fps) to round(to x fps).
- For 9:16 the resolution setting is the short side: `1080p` gives 1080x1920, `720p` gives 720x1280. `--scale` multiplies it.
- The plugin draws captions only when `config.captions` is true (3 to 4 words, uppercase, current word highlighted). `fadeIn` and `fadeOut` default to fades from and to black (0.5 s and 0.8 s). In 9:16 the characters' ground line sits at 68 percent of the height, so the bottom third is floor and foreground.
- `transcribe.py` uses faster-whisper `small.en`, on CUDA only if PyTorch with CUDA is importable, otherwise CPU int8. With a script it aligns the script's words onto Whisper's words and prints the match percentage. **It does not enforce the 90 percent rule in code**; only the skill text asks Claude to. Output: `timeline/words.json` as `{source, words:[{word,start,end}]}`, plus `sentences.json` and `transcript.txt`.
- `beats.py` reads only `config.json` and `timeline/words.json`, so we can write `words.json` ourselves and skip the plugin's transcription.
- `audio/mix.py`: voice + generated or own music + SFX + ambience; music ducked 9 dB and ambience 5 dB under the voice; loudness normalised to `config.loudness` (default -14 LUFS, true peak -1.5); writes `build/mix.wav` and, if the silent video exists, `out/<title>.mp4`. Flags: --video, --out, --no-music, --no-sfx. An SFX cue name can be a relative path to your own wav/mp3. Your own music is one file (`config.music.file`), looped or trimmed.
- `shots.json` fields: id, start, end, line, scene, set, bars, in {type cut|fade|dip|dipWhite|flash|whip|zoom, dur, dir}, sfx[], amb, params, vignette, captions, notes; top level: title, duration, shots[], music[] sections.

- The plugin's own sound guide calls generated music a serviceable bed and suggests your own licensed track for premium results.

18.2 Assumed, not verified (tests in Part 26)

- Speed on your laptop, and how the plugin behaves with the RTX 3050 4 GB in GPU mode.
- Whether extra keys in `words.json` or `shots.json` are tolerated.
- Where the player loads words from (assumed `timeline/words.json`).
- Whether the first shot of a range render blends with the previous shot (from reading `player.js`, transitions draw the previous shot's frame).
- The sample rate of `mix.wav`, and how our Python environment coexists with the plugin's tools.
- How a `.plugin` file installs into Claude Code (we avoid needing this by vendoring the kit).
- Your rights to use the plugin commercially and to monetise its output: **unknown**.

18.3 Where I changed your design

1. **Option A first, as you said, with one change: the plugin owns pictures only; our pipeline owns audio.** Your SFX and music folder is part of the brand sound; the plugin accepts only one own music file; Remotion segments need SFX cues too; one cue list gives one mix. The plugin's `mix.py` is used once in the first test as a sanity check, then only as a fallback.
2. **Plugin captions off, fadeIn and fadeOut 0, bars none.** Remotion owns captions and every fade.
3. **One alignment run.** Our `align.py` writes `timeline/words.json` in the plugin's format. We skip the plugin's `transcribe.py` and call `beats.py` on our file.
4. **Style is chosen once, not per video.** We add one custom style to `studio/engine/styles.js` in the brand step and skip the plugin's per-video 5-style grid.
5. **Do not run the plugin's `make-2d-video` skill end to end inside our orchestrator.** It is interactive (intake questions, three checkpoints) and wants to own the workflow. We vendor the kit and its reference docs and write thin skills of our own that call the kit's tools.

6. **Split work by strength.** Plugin: characters, cinematic scenes, 3D, simple numbers and blueprints (it has its own data cards, bars and meters). Remotion: maps, timelines, real data charts, photos and footage, logos, end card, captions, brand motion. Do not rebuild in Remotion what the plugin already does well, and do not ask the plugin for maps.
7. **Unify the look.** One `brand/palette.json` feeds both the plugin's custom grade and the Remotion theme; Remotion takeovers use the same vignette and grain overlay and the brand transitions.
8. **Use 30 fps on both sides.** It costs about 25 percent more plugin frames than the 24 fps default, but mixing a 24 fps clip into a 30 fps composition causes uneven frame repeats. Test 24 against 30 once (Part 26).
9. **Brand sameness is the strategic risk.** The plugin's visual style is fixed and shared by everyone who has it. Your identity must come from the custom grade, set library, casting conventions, caption style, voice, sound, structure and the Remotion layer. Keep a non-stick-figure format in rotation so the channel is not one-note.
10. **The hidden cost is Claude usage, not render time.** The plugin's shot-building stage has Claude write JavaScript for 15 to 25 shots per minute and inspect contact sheets. On Pro this is probably what limits 20 to 30 videos a month. Mitigations: a reusable set and prop library, shot recipes, 30 to 45 second videos, a Sonnet subagent for shot code, capped QC loops. Measure after the test video.

19. Pipeline v2, stage by stage

Each video has `videos/<id>/` (ours) and a generated plugin project at `studio/projects/<id>/`. `state.json` records each stage's status and input hashes. The plugin's three checkpoints are folded into our gates: setup happens once, style happens once (brand step), preview is Gate 4.

- **P0 Intake** (you, `/ch-new`): title or idea. Produces `00_brief.md`, `state.json`.
- **P1 Research** (`ch-research`, subagent `researcher`): brief in; `01_research.md`, `sources.json` out. **Gate 1: you approve.**
- **P2 Script** (`ch-script`, then subagent `fact-checker`): research and brand voice in; `02_script.md` (3 hook options, scene split), `script.txt` (plain narration, no headings), pronunciation updates, `factcheck.md` out. **Gate 2: you approve.** After this the narration is frozen; the plugin's rule is never to alter it.
- **P3 Voice** (`pipeline/tts.py`, Kokoro): `script.txt` + `brand/voice.json` in; `audio/voice.wav` (48 kHz), `audio/voice_parts/` out.

- **P4 Timing** (`pipeline/align.py` , faster-whisper with script as truth; then the plugin's `beats.py`): `voice.wav` + `script.txt` in; `timeline/words.json` (plugin format), `timeline/words.frames.json` (Remotion copy), `timeline/align_report.json` , `timeline/beats.json` , draft `story/shots.json` out. The pipeline stops if the script match is below 95 percent.
- **P5 Direction** (`ch-storyboard` , subagent `director`): script, beats, brand, component catalog, SFX catalog in; `master.json` (segments, overlays, cues), `story/bible.md` , `scenes/cast.js` , `03_assets_needed.md` , `qa/sfx_audition.html` out. **Gate 3: you approve the storyboard, the plugin/Remotion mix, the asset list and the SFX picks.**
- **P6 Assets** (`ch-assets`): asset list in; files in `library/` , records in `licenses.json` ; `stage_assets.py` copies what is needed into `remotion/public/staged/<id>/` and `studio/projects/<id>/input/` . `validate_assets.py` must pass.
- **P7 Plugin scenes** (`ch-2d-shots` , subagent `shot-builder`): shots, bible, cast in; `scenes/sets/*.js` , `scenes/shots/*.js` ; contact sheets via `render.mjs --sheet --scale 0.5` ; QC-1 loop.
- **P8 Plugin render** (`pipeline/render_plugin.py`): Mode A renders the full silent video; Mode B renders one clip per plugin segment with `--from/--to --out` , with handle frames. For the first three videos, render a 10 second preview range first.
- **P9 Mix** (`pipeline/mix_audio.py`): `voice.wav` , `master.json` cues, library SFX and music in; `audio/mix.wav` (-14 LUFS) out.
- **P10 Remotion build** (`ch-remotion-build`): `master.json` , `words.frames.json` , staged assets in; compositions, QC-2 stills, draft render at scale 0.5.
- **P11 QA** (`ch-qa` , subagent `qa-reviewer`): Part 24. `qa/report.json` out.
- **Gate 4: you watch and listen to the draft.**
- **P12 Revise** (`ch-revise`): `feedback_vN.md` in; a minimal re-run plan out (Part 24.4).
- **P13 Final** (`ch-publish`): full-resolution render, `publish.md` , copy to `exports/` .
- **P14 Log**: row in `analytics/experiments.csv` .

The orchestrator `ch-run` walks these stages, stops at each gate, and resumes from `state.json` after any interruption.

20. Shared data contract

Rules:

1. **Time is integer frames at the project fps (30).** Seconds appear only in plugin files and `words.json`, derived as frame / fps. All boundaries are snapped to the frame grid before plugin files are written.
2. **`master.json` is the single source of truth** for segment timing, overlays and audio cues. The plugin's `config.json` and the timing fields of `story/shots.json` are generated from it. Scene descriptions stay editable in `shots.json`.
3. **One alignment run.** `timeline/words.json` keeps the plugin's exact format. `timeline/words.frames.json` is the Remotion copy. Neither is edited by hand.
4. **One mix.** Plugin shots keep `sfx` and `amb` empty; all cues live in `master.json.audio.cues`.
5. **Stable IDs.** Segment, shot, cue and asset ids never change after Gate 3 without a migration note.
6. **Handles.** In Mode B each plugin clip renders N extra frames (default 9) before and after its segment so Remotion transitions have picture on both sides.

20.1 `schemas/master.schema.json` (condensed)

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "ch.master/1",
 "type": "object",
 "required":
["schema","id","title","fps","width","height","durationFrames","mode","voice"
,"words","segments","audio"],
 "properties": {
  "schema": {"const": "ch.master/1"},
  "id": {"type": "string"},
  "title": {"type": "string"},
  "fps": {"enum": [24, 30]},
  "width": {"const": 1080},
  "height": {"const": 1920},
  "durationFrames": {"type": "integer", "minimum": 1},
  "mode": {"enum": ["A", "B"]},
  "voice": {"type": "object", "required": ["file","voiceId","speed"],
    "properties": {"file": {"type": "string"}, "voiceId": {"type": "string"},
"speed": {"type": "number"}}},
  "words": {"type": "object", "required": ["pluginFile","framesFile"],
    "properties": {"pluginFile": {"type": "string"}, "framesFile": {"type":
"string"}}},
  "segments": {"type": "array", "minItems": 1, "items": {"$ref":
```

```
"#/$defs/segment"}},
  "overlays": {"type": "array", "items": {"$ref": "#/$defs/overlay"}},
  "captions": {"$ref": "#/$defs/captions"},
  "audio": {"$ref": "#/$defs/audio"},
  "attributions": {"type": "array", "items": {"type": "string"}}
 },
 "$defs": {
  "frames": {"type": "integer", "minimum": 0},
  "segment": {"type": "object", "required":
["id","type","startFrame","endFrame"],
   "properties": {
    "id": {"type": "string"},
    "type": {"enum": ["plugin", "remotion"]},
    "startFrame": {"$ref": "#/$defs/frames"},
    "endFrame": {"$ref": "#/$defs/frames"},
    "line": {"type": "string"},
    "transitionIn": {"type": "object", "properties": {"type": {"enum":
["cut","fade","dip","wipe","zoom","flash"]}, "frames": {"type": "integer"}}},
    "plugin": {"type": "object", "required":
["clip","clipOffsetFrames","shotIds"],
      "properties": {"clip": {"type": "string"}, "clipOffsetFrames": {"$ref":
"#/$defs/frames"}, "shotIds": {"type": "array", "items": {"type":
"string"}}}},
    "remotion": {"type": "object", "required": ["component","props"],
      "properties": {"component": {"type": "string"}, "props": {"type":
"object"}, "assets": {"type": "array", "items": {"type": "string"}},
"hideCaptions": {"type": "boolean"}}}
   },
   "allOf": [
    {"if": {"properties": {"type": {"const": "plugin"}}}, "then":
{"required": ["plugin"]}},
    {"if": {"properties": {"type": {"const": "remotion"}}}, "then":
{"required": ["remotion"]}}
   ]},
  "overlay": {"type": "object", "required":
["id","kind","startFrame","endFrame"],
   "properties": {"id": {"type": "string"}, "kind": {"enum":
["hookText","lowerThird","logoBug","progressBar","endCard","sourceTag"]},
    "startFrame": {"$ref": "#/$defs/frames"}, "endFrame": {"$ref":
"#/$defs/frames"}, "props": {"type": "object"}}},
  "captions": {"type": "object", "properties": {"enabled": {"type":
"boolean"}, "style": {"type": "string"}, "maxWords": {"type": "integer"},
   "safe": {"type": "object", "properties": {"top": {"type": "integer"},
```

```
"bottom": {"type": "integer"}, "side": {"type": "integer"}}}}},
  "audio": {"type": "object", "required":
["mixFile","loudnessTarget","cues"],
   "properties": {"mixFile": {"type": "string"}, "loudnessTarget": {"type":
"number"},
    "music": {"type": "array", "items": {"type": "object", "required":
["trackId","startFrame","endFrame"],
     "properties": {"trackId": {"type": "string"}, "startFrame": {"$ref":
"#/$defs/frames"}, "endFrame": {"$ref": "#/$defs/frames"}, "gainDb": {"type":
"number"}}}},
    "cues": {"type": "array", "items": {"type": "object", "required":
["id","frame","sfxId"],
     "properties": {"id": {"type": "string"}, "frame": {"$ref":
"#/$defs/frames"}, "sfxId": {"type": "string"}, "gainDb": {"type": "number"},
"segmentId": {"type": "string"}}}}}}
 }
}
```

Extra validator rules the schema cannot express (`validate.py` enforces them): segments are contiguous and cover frame 0 to durationFrames; every plugin segment's shots exist in `story/shots.json`; every `sfxId`, `trackId` and asset id exists in the catalogs and `licenses.json`; every component name exists in the Remotion registry and its props pass that component's own schema.

20.2 Example `master.json` (abridged)

```
{
 "schema": "ch.master/1", "id": "2026-11-03_demo", "title": "Demo", "fps":
30, "width": 1080, "height": 1920,
 "durationFrames": 1290, "mode": "A",
 "voice": {"file": "audio/voice.wav", "voiceId": "<kokoro-voice-id>",
"speed": 1.0},
 "words": {"pluginFile": "timeline/words.json", "framesFile":
"timeline/words.frames.json"},
 "segments": [
  {"id": "seg01", "type": "plugin", "startFrame": 0, "endFrame": 240, "line":
"...",
   "plugin": {"clip": "staged/plugin_full.mp4", "clipOffsetFrames": 0,
"shotIds": ["s001","s002","s003"]}},
  {"id": "seg02", "type": "remotion", "startFrame": 240, "endFrame": 420,
"transitionIn": {"type": "wipe", "frames": 9},
   "remotion": {"component": "MapPanZoom", "props": {"focus": "...",
```

```
"labels": ["..."]}, "assets": ["map_natural_earth"]}},
  {"id": "seg03", "type": "plugin", "startFrame": 420, "endFrame": 1290,
   "plugin": {"clip": "staged/plugin_full.mp4", "clipOffsetFrames": 420,
"shotIds": ["s010","s011"]}}
 ],
 "overlays": [
  {"id": "o1", "kind": "hookText", "startFrame": 0, "endFrame": 75, "props":
{"text": "..."}},
  {"id": "o2", "kind": "progressBar", "startFrame": 0, "endFrame": 1290},
  {"id": "o3", "kind": "endCard", "startFrame": 1230, "endFrame": 1290}
 ],
 "captions": {"enabled": true, "style": "brand_default", "maxWords": 3,
"safe": {"top": 150, "bottom": 250, "side": 60}},
 "audio": {"mixFile": "audio/mix.wav", "loudnessTarget": -14,
  "music": [{"trackId": "mus_007", "startFrame": 0, "endFrame": 1290,
"gainDb": -18}],
  "cues": [{"id": "c001", "frame": 42, "sfxId": "sfx_0142", "gainDb": -6,
"segmentId": "seg01"}]},
 "attributions": []
}
```

20.3 Derived files

- `timeline/words.json` (plugin format, exactly): `{"source": "align:faster-whisper-small+script", "words": [{"word": "Hello", "start": 0.133, "end": 0.4}]}`.
- `timeline/words.frames.json` (Remotion): `{"fps": 30, "words": [{"i": 0, "text": "Hello", "startFrame": 4, "endFrame": 12, "segmentId": "seg01"}]}`.
- Plugin `config.json`, generated: `{"title": "...", "aspect": "9:16", "resolution": "1080p", "fps": 30, "style": "<brand-style-id>", "voiceover": "input/voiceover.wav", "script": null, "duration": <seconds>, "captions": false, "bars": null, "fadeIn": 0, "fadeOut": 0, "music": {"enabled": false}, "sfx": {"enabled": false}, "loudness": -14}`.
- Plugin `story/shots.json`: contiguous shots covering 0 to duration (the renderer needs every moment covered). In Mode B, Remotion-owned time ranges get one placeholder shot each (scene: 'skip') and are never rendered, because only plugin ranges are passed to `--from/--to`.

21. Folder structure v2

```
C:/channel
  CLAUDE.md                     # short; pointers to docs and the v2 rules in
Part 23.5
  .claude/
    skills/                     # ch-run, ch-new, ch-research, ch-script, ch-
voice, ch-storyboard,
                                #   ch-assets, ch-2d-shots, ch-audio, ch-
remotion-build, ch-qa,
                                #   ch-revise, ch-publish, ch-brand, ch-queue
    agents/                     # researcher.md, fact-checker.md,
director.md, shot-builder.md, qa-reviewer.md
  docs/                         # MASTER_PLAN.md, vendor-notes.md (kit
version, hash, local changes)
  brand/                        # brand.md, palette.json, voice.json,
pronunciations.json, components.md, fonts/
  config/pipeline.json
  schemas/                      # master, words.frames, licenses, catalog,
sources, state
  pipeline/                     # v1 scripts plus make_plugin_files.py,
render_plugin.py, build_props.py,
                                #   qa_stills.py, run_queue.py
  studio/                       # vendored plugin kit (pinned, git-tracked,
node_modules ignored)
    render.mjs  engine/  tools/  audio/  examples/  package.json
    shared/sets/  shared/props/  shared/recipes/   # OUR reusable sets, props
and shot recipes
    projects/<video-id>/        # generated plugin project: config.json
input/ timeline/ story/ scenes/ build/
  remotion/
    src/components/{segments,overlays,captions,transitions}  src/theme
src/Root.tsx
    public/staged/<video-id>/
  library/                      # sfx, music, logos, footage, images, maps,
licenses.json
  videos/<yyyy-mm-dd_slug>/     # 00_brief.md 01_research.md sources.json
02_script.md script.txt factcheck.md
                                #   master.json timeline/ audio/ qa/ renders/
feedback_vN.md publish.md state.json
  exports/  analytics/  cache/  logs/  tests/  .venv/
```

Vendoring rules: copy the plugin's `kit/` and `references/` into `studio/` and `docs/vendor/` once, record the version and file hashes in `docs/vendor-notes.md`, never edit kit files except `engine/styles.js` (our custom style), and log every local change. Do not install the plugin's own skills into the project at the same time, so there are no name clashes with our `ch-` skills.

22. Remotion composition structure

Amendment to Part 19 (order change): run P3 (voice) and P4 (alignment) before Gate 2, so that Gate 2 means *script and voice.wav approved together*. Pronunciation or wording fixes after this point change every duration and force a full plugin re-render; fixing them here costs a few seconds of TTS. After Gate 2 the narration and timing are frozen.

22.1 Data in, one props file

Remotion code runs in a browser and cannot read arbitrary files, so `pipeline/build_props.py` merges `master.json` and `timeline/words.frames.json` into one props file, rewrites asset paths relative to `remotion/public/`, and the render is started with `npx remotion render ShortVideo out.mp4 --props=<file>`. The composition `ShortVideo` takes its size, fps and duration from the props (`calculateMetadata`; check the current name and signature in the Remotion docs).

22.2 Layers, bottom to top

- **L0 Background:** brand solid; visible only if a clip has a gap.
- **L1 PluginLayer:** one `Sequence` per plugin segment, each playing an `OffthreadVideo` of the plugin clip, muted, started at the segment's `clipOffsetFrames`. In Mode A every segment points to the same file with a different offset. The prop that sets the start offset is `startFrom` in older Remotion and may be `trimBefore` in newer versions; check the docs.
- **L2 SegmentLayer:** one `Sequence` per Remotion segment, starting `transitionIn.frames` early so it overlaps the previous segment, rendered through `TransitionWrap` (brand wipe, zoom, fade or dip, animated with the brand easing curves). Components come from the registry (22.3). A `GradeOverlay` (vignette and grain matching the plugin's look) sits above L1 and L2.
- **L3 Overlays:** hookText, lowerThird, logoBug, sourceTag, progressBar, endCard, in that z-order.

- **L4 Captions:** the caption component (22.4).
- **L5 Debug:** safe-zone guides, enabled only for QA stills through a `debug` prop.
- **Audio:** exactly one `Audio` element at the root playing `audio/mix.wav`. All video elements are muted.

```
// remotion/src/ShortVideo.tsx (sketch; verify prop names against current
Remotion docs)
export const ShortVideo = ({master, words, debug}) => (
  <AbsoluteFill style={{background: theme.bg}}>
    {master.segments.filter(s => s.type === 'plugin').map(s => (
      <Sequence key={s.id} from={s.startFrame} durationInFrames={s.endFrame -
s.startFrame}>
        <OffthreadVideo src={staticFile(s.plugin.clip)} startFrom=
{s.plugin.clipOffsetFrames} muted />
      </Sequence>
    ))}
    {master.segments.filter(s => s.type === 'remotion').map(s =>
<SegmentRenderer key={s.id} seg={s} />)}
    <GradeOverlay />
    {master.overlays.map(o => <OverlayRenderer key={o.id} o={o} />)}
    {master.captions.enabled && <Captions words={words} cfg={master.captions}
segments={master.segments} />}
    {debug && <SafeZones cfg={master.captions.safe} />}
    <Audio src={staticFile(master.audio.mixFile)} />
  </AbsoluteFill>
);
```

22.3 Segment registry

`remotion/src/components/registry.ts` maps a component name to its React component, a props schema (zod or similar), a one-line description, minimum and maximum duration in frames, and a `hidesCaptions` flag. `brand/components.md` is generated from the registry so the storyboard skill only uses components and props that exist. First set: MapPanZoom, Timeline, BarChart, StatCounter, ImageSlowZoom, FootageClip, ComparisonSplit, QuoteCard, NameReveal (logo or wordmark), SourceTag. Overlays: HookText, LowerThird, LogoBug, ProgressBar, EndCard.

22.4 Caption component

- Reads `words.frames.json`; the text is the script's own spelling because `align.py` keeps script words and only borrows Whisper's timing.

- Groups words into phrases: at most 3 words by default; breaks at punctuation or a gap over 8 frames; never splits a number from its unit or a two-word name.
- Highlights the active word in the brand accent with a short spring pop; phrases fade in and out over 3 frames.
- Measures text (layout-utils or equivalent) so a phrase fits the width inside the safe zone and never exceeds two lines.
- Default position: centred at about y = 1500 on the 1080x1920 canvas, inside the safe zone (side 60, top 150, bottom 250 as starting values; tune on real phones). This is below the plugin's characters, whose ground line is at 68 percent of the height.
- Hidden during segments flagged `hideCaptions`; moves up 120 px while a lowerThird is active.
- No `Math.random`, no `Date.now`. Exposes a list of stress frames (the longest phrases) for QC stills.

22.5 Inserting plugin clips

- **Mode A:** one full-length silent video from the plugin, played in segments with offsets. Remotion segments cover the plugin picture beneath them, so those plugin frames were rendered for nothing; Mode B fixes that.
- **Mode B:** `render_plugin.py` renders one clip per plugin segment with `--from (start - handle) / fps --to (end + handle) / fps --out <file>`, handle = 9 frames each side, so `clipOffsetFrames` = 9. Write the times with 6 decimals so round(time x fps) lands on the intended frame.
- Force the first shot of every range to `in: cut` (see Part 26, R3) unless the handle test shows a clean transition.

23. Skills and subagents

23.1 Where they live

Project scope: skills in `.claude/skills/<name>/SKILL.md`, subagents in `.claude/agents/<name>.md`. Both are committed to git with the project. Per the Claude Code docs, skills at several scopes resolve in the order enterprise, personal, project when names clash, so every name here carries the `ch-` prefix, and the plugin's own skills (make-2d-video and others) are not installed in the project. A subagent file has frontmatter with `name`, `description`, optional `tools` (inherits all if omitted) and

optional `model` (`sonnet`, `opus`, `haiku` or `inherit`); each subagent works in its own context window. Verify exact field names with `/agents` and the docs before generating the files.

Design choice: the main conversation drives the gates (`ch-run`); subagents do bounded work and return a short summary plus file paths, which keeps the main context small.

23.2 Skills

Every stage skill sets `disable-model-invocation: true`, so it runs only when you or `ch-run` call it. Every skill follows one template: Inputs, Outputs, Done when, Stop and ask when. Every skill starts by reading `state.json` and ends by updating it.

- **ch-run (orchestrator).** In: video id, optional target stage. Reads `state.json`; runs the next stages in order by calling the other skills, subagents and scripts; stops at each gate with a five-line summary, the files to review, open issues, and the exact words to approve or request changes. Records approvals in `state.json` and tags git (`gate1-<id>` and so on). Never skips a gate; reopens a gate if a frozen input must change. Resumes at the first incomplete stage after a crash or usage-limit stop. Out: `state.json`, `logs/<id>.md`.
- **ch-new.** Creates `videos/<id>/`, the brief template and `state.json`.
- **ch-brand** (one time, then on change). Voice audition, style exploration, the custom plugin style in `studio/engine/styles.js`, palette sync to Remotion, caption style. Out: `brand/*`, an entry in `docs/vendor-notes.md`.
- **ch-research** (runs subagent `researcher`). In: brief. Out: `01_research.md`, `sources.json`. Rules from Part 3.
- **ch-script** (then subagent `fact-checker`). In: research, brand voice. Out: `02_script.md`, `script.txt`, `factcheck.md`, pronunciation updates.
- **ch-voice.** Runs `tts.py`, `align.py`, `beats.py`. Out: `audio/voice.wav`, `timeline/*`. Stops if the script match is below 95 percent.
- **ch-storyboard** (subagent `director`). In: script, beats, brand, `brand/components.md`, SFX catalog. Out: `master.json`, `story/bible.md`, `scenes/cast.js`, shot skeleton, `03_assets_needed.md`, `qa/sfx_audition.html`. Rules: plugin carries roughly 65 to 85 percent of the duration; Remotion takes maps, charts, photos, footage and logos; at most one layer switch every five seconds or so (a starting guess, tune it by taste); a plugin or hook-text picture in the first 3 seconds; end card in the last 1.5 to 2 seconds; SFX only from the core set.

- **ch-assets.** As Part 8. Out: library files, `licenses.json`, staged copies, passing `validate_assets.py`.
- **ch-2d-shots** (subagent `shot-builder`). Wraps the plugin's knowledge: reads the vendored `style-bible`, `engine-api`, `story-breakdown` and `render-qc` references once per session; builds sets and shots in batches of 8 to 12; reuses `studio/shared/` first and promotes good new sets there; obeys our rules (no captions, bottom-third safe zone, `V()` for every aspect-dependent value, no `Math.random` or `Date.now`, 30 fps). Out: `scenes/*`, `build/sheets/*.png`.
- **ch-audio.** Runs `mix_audio.py`; checks every cue against the timeline; Out: `audio/mix.wav`, `audio/cues.json`.
- **ch-remotion-build.** Uses the official Remotion skill for API details. Builds props, composes, renders QC stills and the draft.
- **ch-qa** (subagent `qa-reviewer`). Part 24.
- **ch-revise.** Classifies `feedback_vN.md` and writes the minimal re-run plan (24.5).
- **ch-publish.** Final render, `publish.md` (titles, description, hashtags, attribution lines, disclosure decision), `exports/`.
- **ch-queue** (later). Runs approved videos through P8 to P13 overnight, stops on the first error, writes a morning report.

23.3 Subagents

- **researcher** (`model: inherit`; tools: web search, web fetch, Read, Write, Grep). Works from the brief, writes the two research files, returns a summary of at most 20 lines.
- **fact-checker** (`inherit`; Read, Grep, web fetch). Checks every factual sentence in `script.txt` against `sources.json`; writes `factcheck.md` marking each claim supported, unsupported or uncertain.
- **director** (`inherit`; Read, Write, Edit, Bash). Builds `master.json` and the storyboard files; runs `validate.py` before returning.
- **shot-builder** (`sonnet`; Read, Write, Edit, Bash). At most 12 shots per call; returns the files written and the sheet paths.
- **qa-reviewer** (`sonnet`; Read, Bash, Glob, Grep). Reads contact sheets and stills, writes only inside `qa/`.

23.4 One thing the orchestrator must know

The plugin's own docs have Claude ask the user three times (setup, style, preview). Our `ch-run` never delegates to those prompts; its gates replace them.

23.5 `CLAUDE.md` additions for v2

```
## Pipeline v2 rules
- Time is integer frames at 30 fps. master.json is the only timing source.
Never hand-edit words.json or words.frames.json.
- Plugin (studio/) draws pictures only: captions false, fadeIn 0, fadeOut 0,
sfx and music disabled, no shot-level sfx or amb.
- Remotion owns captions, fades, transitions between segments, overlays and
the final audio (one Audio element, audio/mix.wav).
- Do not run the plugin's make-2d-video skill. Use ch-2d-shots and the
vendored references.
- Do not edit studio/ kit files except studio/engine/styles.js; log changes
in docs/vendor-notes.md.
- Narration is frozen after Gate 2 (script and voice). Segment types are
frozen after Gate 3.
- Reuse studio/shared/ sets, props and recipes before writing new ones.
- Run pipeline/validate.py before every render.
```

24. Automatic QC loop

24.1 Levels

- **QC-0 Before any render (script, seconds):** `validate.py` (schema plus the extra rules in 20.1), frame-grid check, words match, assets licensed and staged, fonts present, props validate against each component's schema, a one-frame Remotion smoke render.
- **QC-1 Plugin contact sheets (P7):** after each batch of 8 to 12 shots run `render.mjs --sheet --scale 0.5` and open every `build/sheets/sheet_XX.png`. Check: no `MISSING SHOT` or red ERROR frames and no shot errors in the render log; each picture matches its line and the key action lands on the key word; characters readable with rim light and not cut off; consistent outfits and screen direction; moody value structure and brand grade; every shot has camera motion and atmosphere; on-screen text short, legible at phone size, and numbers match the narration; variety (no more than 3 similar shots in a row); **our additions:** nothing important in the top 8 percent or the bottom 24 percent of the frame, no plugin-drawn captions, and no plugin text that Remotion will also show.

- **QC-2 Remotion stills (P10):** `remotion still` at the first frame, both sides of every segment boundary, the middle of every segment, every overlay's first and last frame, the caption stress frames and the end card, with the safe-zone debug layer on. Check for overflow, overlaps, caption collisions with overlays, wrong asset, empty frame, and visual jump at plugin/Remotion seams (colour, grain, black flash).
- **QC-3 Final file (P11):** ffprobe facts (1080x1920, constant 30 fps, H.264, yuv420p, AAC 48 kHz, duration within one frame of the timeline), loudness near -14 LUFS with true peak at or below -1 dBTP, no unplanned silence or black frames, caption timing compared with `words.frames.json`, attribution lines present.
- **QC-4 Cross-layer:** count of plugin/Remotion switches; no double information (plugin data card and Remotion stat showing the same number); transitions land on cue frames.

24.2 Fix automatically (up to 3 loops per issue)

Text overflow or wrap, safe-zone breaches, caption wrapping, missing staging, cue timing off by fewer than 3 frames, loudness out of range, shot errors and red or MISSING frames, flicker from non-seeded randomness, wrong size or fps, a missing fade or handle, a prop that fails its schema.

24.3 Fix, then report at Gate 4

Small composition changes (camera move, prop placement, easing), choosing between two valid transitions, trimming a segment by a few frames.

24.4 Always ask you

Any change to meaning (wording, a fact, a number); swapping a segment's type or concept; anything touching palette, voice, caption style or brand; any asset with an unclear license; a depiction of a real person, company or event that could mislead; script-to-audio match below 95 percent; any issue still open after 3 loops; and any taste call between two reasonable looks. Each open item goes into `qa/report.json` with severity (blocker, major, minor), the loops used and what was tried.

24.5 Revision routing (`ch-revise`)

- Wording, fact or voice change: reopen Gate 2, then everything downstream re-runs. This is why Gate 2 includes the voice.

- Visual change inside a plugin shot: edit that scene file, re-render only that range (Mode B) or the full video (Mode A), then P9 to P11.
- Remotion-only change (captions, overlay text, segment props, transition): re-run P10 and P11 only.
- SFX or music change: re-run P9, then mux the new `mix.wav` onto the existing render with ffmpeg stream copy for the video; no re-render is needed for audio-only changes.
- Segment type change (plugin to Remotion or back): reopen Gate 3.
- Cue or segment timing shift under 6 frames: edit `master.json` and re-run P9 to P11.

25. Render-time and resource budget

Basis: an average video of 40 seconds (1,200 frames at 30 fps), 1080x1920, Mode A. **Every figure below is an estimate; none was measured on your laptop.** The plugin author measured about 2 fps at 720p on a 2-thread cloud CPU, 1080p about 2 times slower, and says a modern desktop is 3 to 5 times faster. The first benchmark (Prompt 5) replaces these guesses with real numbers.

25.1 Time per video

- **Plugin render:** assume 2 to 5 fps at 1080x1920, so 1,200 frames take about 4 to 10 minutes; a pessimistic 1 fps is 20 minutes. At 24 fps the same video has 960 frames, about 20 percent less.
- **Remotion draft and final:** assume 4 to 10 fps with a full-frame video base, captions and overlays; about 2 to 5 minutes per full render. A draft at scale 0.5 is roughly half of that or less.
- **Voice and alignment:** Kokoro and Whisper small on CPU, about 1 to 3 minutes for 40 seconds of speech.
- **Mix and QA stills:** about 1 to 2 minutes.
- **Per finished video:** about 10 to 25 minutes of compute. With a preview, a draft and one revision cycle, plan 15 to 40 minutes.
- **At 20 to 30 videos a month:** about 5 to 20 hours of compute, which fits in overnight queues. Mode B saves roughly the share of the video that Remotion owns (30 percent Remotion segments, about 30 percent less plugin time).

25.2 Memory, GPU, disk

- **RAM:** each plugin worker is a separate Chromium, about 0.6 to 1.5 GB at 1080p per the plugin's docs. The default is half the CPU threads (a 12-thread CPU means 6 workers, possibly 4 to 9 GB). Start with `--workers 3` on 16 GB and 2 on 8 GB. Remotion concurrency 2. Never run the two renderers at the same time.
- **GPU (RTX 3050, 4 GB):** `--gl auto` uses the GPU on Windows, which helps the 3D shots most. If 3D shots come out black or Chromium crashes, switch to `--gl swiftshader` (CPU, slower). The benchmark decides.
- **Disk:** the plugin pipes frames straight to ffmpeg, so there are no frame folders. Expect about 100 to 200 MB per video including draft and final, so 4 to 6 GB a month. Playwright's Chromium and Remotion's browser are separate installs; keep 60 GB free.
- **Heat:** plugged in, best-performance mode, cooling pad, sleep off during queues.

25.3 Ways to cut cost

- Render only what is needed: preview ranges for the first three videos, `--from/--to` for fixes, Mode B so plugin frames under Remotion takeovers are never drawn.
- Drafts at `--scale 0.5` and `--format jpeg` (the plugin says jpeg capture is about 1.5 times faster with negligible loss); decide by test whether finals can use jpeg too.
- Keep videos at 30 to 45 seconds; fewer shots, fewer frames.
- Cache voice, alignment and staged assets by hash; skip stages whose inputs did not change.
- Avoid large blur amounts in plugin scenes; blur cost grows with resolution.
- Reuse sets from `studio/shared/` so scene code and sheet checks shrink over time.
- Queue approved videos overnight with `ch-queue`; the queue is resumable and isolates each video.
- Try 24 fps for both tools only if the test shows no visible loss in captions and motion.

25.4 Claude usage (the likely real limit)

My ranking of heavy stages: P7 shot building and QC-1 sheet reviews (code writing plus images) first, then P1 research, then P5 storyboard, then P10, then P2. I cannot turn that into a number. After the test video, read Settings > Usage, multiply by your monthly target, and decide between fewer videos, more reuse, or a higher plan. Start with 10 to 15 videos a month regardless; it protects quality as well as usage.

26. Risks, open questions and how to test them

Status words: VERIFIED means I read it in the files; ASSUMED means a reasoned guess to be tested; UNKNOWN means I cannot tell from here.

- **R1 Plugin license and rights: UNKNOWN.** The package contains no license text. Before you monetise anything, get written terms from the author or seller covering commercial use of the output, use of the kit inside your own pipeline, and modification of `engine/styles.js`. If the answer is no, fall back to pipeline v1 and treat the plugin's look as inspiration only.
- **R2 Speed and GPU behaviour on your laptop: ASSUMED.** Test: Prompt 5 benchmark (workers 2, 3, 4; gl auto and swiftshader; png and jpeg), recorded in `studio/BENCHMARK.md`.
- **R3 A range render may start with a blend of the previous shot: ASSUMED** (from `player.js`, transitions draw the previous shot's frame). Test: render a range that starts at a shot whose `in` is flash, dip or fade and inspect the first frames. If it blends, force `in: cut` on range-starting shots or rely on handles.
- **R4 Extra JSON keys: ASSUMED tolerated.** Test: add a key to a copy of `words.json` and `shots.json` and render. If it fails, keep derived data in separate files (already the plan for `words.frames.json`).
- **R5 Where the player reads words from: ASSUMED `timeline/words.json`.** Test: with `captions: true` in a test config, change one word in that file and see the caption change.
- **R6 Python environment: ASSUMED fine.** Test: run `doctor.py` with `.venv`'s interpreter; check that numpy, scipy, librosa, torch and kokoro import together. Always call `.venv/Scripts/python.exe` by absolute path from wrappers; the plugin docs say `python`, `python3` or `py -3` depending on OS.
- **R7 Frame-exact ranges: ASSUMED.** Test: render two adjacent ranges, join them, and compare the frame count (ffprobe) and the seam with one full render.
- **R8 30 fps correctness: partly VERIFIED.** The time base is `t = frame / fps`. Risk: constants inside effects that assume 24 fps. Test: render the same shot at 24 and 30 and check that motion takes the same seconds.
- **R9 `mix.wav` sample rate and channels: UNKNOWN.** Irrelevant if our mixer owns audio; matters only for the one comparison in the first test.
- **R10 Two browser installs: ASSUMED fine.** If installs or renders are slow, exclude `studio/node_modules` and `remotion/node_modules` from Defender.

- **R11 Installing a `.plugin` file into Claude Code: UNVERIFIED.** We do not need it; we vendor the kit. If you want the plugin's own commands for manual experiments, check the current Claude Code plugin docs first.
- **R12 Claude usage: ASSUMED heavy.** Test: measure after the test video (25.4).
- **R13 Visual sameness and platform rules: real risk.** YouTube evaluates channels as a whole, and this look is available to every user of the plugin. Mitigate with the custom grade, a varied set library, rotating formats (Remotion-only videos), your own voice and structure; watch retention and any monetisation notices.
- **R14 Literal depiction: real risk.** The plugin's rule to show what is said literally can produce misleading pictures. Stick figures standing in for named real people, or accusations drawn as scenes, can mislead or defame. Keep real persons generic or off screen, label recreations as illustrations, and keep every claim sourced.
- **R15 Version drift.** The plugin is v0.1.0. Pin it, hash it, never auto-update, and re-run the benchmark and test video before adopting a new version. Remotion is pinned by the lockfile.
- **R16 A long queue can crash.** Chromium can die in hour three. The queue resumes per stage, isolates each video and writes a morning report.
- **R17 Remotion API names:** `calculateMetadata`, the start-offset prop of `OffthreadVideo`, zod schemas and the `--props` file flag come from my knowledge, not from this session's checks. Prompt 6 tells Claude to confirm each against remotion.dev before use.

Decisions I made so you did not have to answer questions (change any of them): Option A first; 9:16 at 1080x1920 and 30 fps; plugin captions off; our pipeline owns the mix; style chosen once in the brand step; voice approved together with the script at Gate 2; four gates as in v1.

The one question I cannot assume away: **where did the plugin come from, and do you have its license or terms?**

27. Build order and prompts

27.1 Minimal first test (Option A, 30 to 45 seconds)

1. **T0:** Prompts 1 to 3 of Part 17 are done: `doctor` passes and the v1 test video renders.

2. **T1 Kit smoke test:** vendor the kit, run its `verify_kit.py` and `doctor.py`, and run the example self-test (a contact sheet of the reaction-time example). Pass: no errors, a sheet of finished frames.
3. **T2 Benchmark:** 10 second range at 1080x1920 and 30 fps with different workers, gl modes and formats; tests R2, R3, R4, R7, R8. Pass: you know your fps and settings, and every R test has a result.
4. **T3 Contract and scripts:** schema, `validate.py`, `make_plugin_files.py`, `render_plugin.py`, `build_props.py`, updated `mix_audio.py`.
5. **T4 Content:** neutral 90-word script with no real-world claims, Kokoro voice, `words.json` in plugin format, `beats.py`, 8 to 12 shots written with the vendored references, sheets checked (QC-1).
6. **T5 Plugin render:** full silent video, 1080x1920, 30 fps.
7. **T6 Audio:** our mix with your SFX and music; compare once against the plugin's own `mix.py` output as a sanity check.
8. **T7 Remotion:** base video, captions, hook text, progress bar, end card and one takeover segment; QC-2 stills; draft render.
9. **T8 Review:** QC-0 to QC-4, then Gate 4 with you. `TEST_REPORT_V2.md` records time per stage, memory, failures and a go or no-go for R1 to R12.

27.2 Then add, one step at a time

1. Brand step (`ch-brand`): custom plugin style, palette sync, caption look, intro and end card.
2. Component library to about 10 segment types, each proven with stills.
3. Shared library: 6 to 8 reusable sets, props and shot recipes in `studio/shared/`.
4. Skills and subagents (Prompt 7).
5. Mode B for videos with at least 30 percent Remotion segments.
6. Revision tooling: range re-renders and audio-only remux.
7. Queue runner and overnight batches.
8. Analytics log and style experiments (Part 11).
9. Your first 3 real videos through every gate; fix the pain points.
10. Long-form later: the plugin supports 16:9 and longer videos, but its own estimate for long, high-resolution videos is many hours of rendering, so plan per-chapter renders, much more disk, and an overnight workflow.

27.3 Prompt 5: vendor the kit and benchmark

```
Read CLAUDE.md and docs/MASTER_PLAN.md Parts 18 to 21, 25 and 26. Task:
vendor the aiTrends 2D Studio kit and benchmark it. Do not use or install the
plugin's own skills.

1. The plugin archive is at docs/vendor/aitrends-2d-studio.plugin (it is a
zip). List its contents without running anything from it. Copy skills/make-
2d-video/kit/ into studio/ and the references folder into
docs/vendor/references/. In docs/vendor-notes.md record version 0.1.0, the
archive's sha256, and the sha256 of every copied file. Note whether the
archive contains any license text (it should not; say so).
2. In studio/: npm install, npx playwright install chromium, install numpy
and scipy into .venv. Run tools/verify_kit.py and tools/doctor.py with
.venv's interpreter. Fix failures; ask before admin steps.
3. Self-test: node render.mjs --project examples/reaction-time --sheet --
scale 0.3. Open the sheet, confirm finished frames and no errors, then delete
examples/reaction-time/build.
4. Benchmark (tests R2, R7, R8): make a 10 second 9:16 1080p 30 fps project
(adapt the example or write a minimal 3-shot project). Render the same range
with --workers 2, 3 and 4, with --gl auto and --gl swiftshader, and with --
format png and jpeg. Record fps, peak RAM and any errors in
studio/BENCHMARK.md. Render the same shot at 24 and 30 fps and compare how
long its motion lasts.
5. Tests R3 and R4: render a range starting at a shot whose 'in' is flash or
dip and inspect the first frames. Add an extra key to copies of words.json
and shots.json and check whether the render still works.
6. Summarise findings and recommended settings in docs/vendor-notes.md. Stop.
```

27.4 Prompt 6: pipeline v2 and test video

```
Read CLAUDE.md, docs/MASTER_PLAN.md Parts 18 to 26, and studio/BENCHMARK.md.
Build pipeline v2 and produce test video 001_v2_test (Mode A, 30 to 45
seconds, 1080x1920, 30 fps).

First confirm against remotion.dev the API names the plan relies on
(calculateMetadata, the OffthreadVideo start-offset prop, props schemas, the
--props file flag) and tell me about any difference.

1. schemas/master.schema.json from Part 20.1 and pipeline/validate.py with
the extra rules.
2. pipeline/: make_plugin_files.py (config.json, input/voiceover.wav, shot
timing from master.json), render_plugin.py (Mode A and B, handles, workers
and gl from config/pipeline.json), build_props.py, mix_audio.py reading
master.json cues, qa_stills.py, a run_queue.py stub.
3. align.py writes timeline/words.json in the plugin's exact format plus
words.frames.json, and fails below 95 percent match.
4. Remotion: ShortVideo per Part 22 with PluginLayer, SegmentLayer,
GradeOverlay, HookText, ProgressBar, EndCard, Captions and the SafeZones
debug layer, plus one segment component (MapPanZoom or StatCounter).
5. Test content: a neutral made-up 90-word script with no real-world factual
claims, a Kokoro voice, my SFX and music via the catalog, 8 to 12 plugin
shots written with the vendored references, one Remotion takeover.
6. Run P3 to P11. Run QC-0 to QC-4 from Part 24, fix what Part 24.2 says to
fix, then stop for me at Gate 4 with qa/report.json and renders/draft_v1.mp4.
7. Write TEST_REPORT_V2.md: time per stage, peak RAM, what broke, what you
assumed, and a go or no-go for each of R1 to R12.
```

27.5 Prompt 7: generate the skills and subagents

```
Read docs/MASTER_PLAN.md Part 23 and TEST_REPORT_V2.md. Create the skills and
subagents listed in Part 23 under .claude/skills and .claude/agents, based on
what actually worked in the test. First verify the frontmatter fields against
the current Claude Code docs. Keep each SKILL.md under 200 lines and put long
reference text in separate files beside it. Set disable-model-invocation:
true on stage skills. Then dry-run ch-run on a video 002 up to Gate 1 without
researching a real topic, and report any problems.
```